MEMORIA PRÁCTICA 4
DISPLAY LCD

Patricia Cárdenas Fernández
3º Computadores A

ÍNDICE

1. MATERIALES Y HERRAMIENTAS..... 2
2. FUNCIONAMIENTO Y MONTAJE..... 2
3. ANOTACIONES DEL CÓDIGO..... 3
3.1. NUESTRAS FUNCIONES DEFINIDAS..... 3
3.2. NUESTRO BUCLE WHILE..... 5

1. MATERIALES Y HERRAMIENTAS

Para realizar esta práctica hemos utilizado un display LCD, cables, resistencias y un transistor. Estos se han utilizado para montar el circuito en una protoboard.

Para tener mayor control sobre el display LCD necesitamos programar en C el comportamiento que queremos. Para ello hemos utilizado un microcontrolador **STM32 nucleo**, el cual lo hemos conectado al circuito.

Además, necesitamos configurar el microcontrolador para que sea capaz de suministrar las señales a los componentes del circuito. El programa **STM32CubeMX** nos facilita la configuración e inicialización de los pines que queremos usar. Una vez configurados los pines del microcontrolador, pasamos a programar el funcionamiento en un entorno de programación, como es **eclipse**.

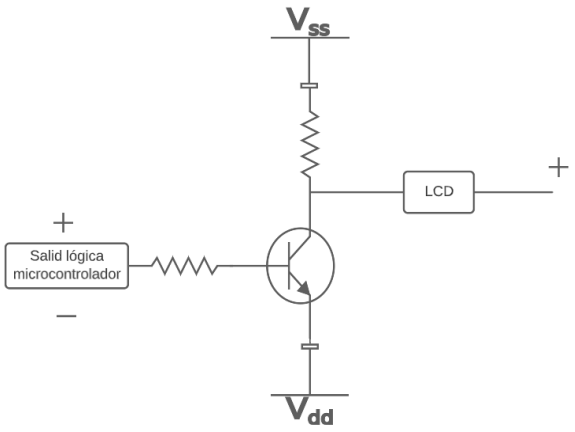
2. FUNCIONAMIENTO Y MONTAJE

El display LCD tiene 8 pines, los cuales se se explican a continuación:

- Vcc: conectado a 5V de la placa.
- Vss: conectado a GND de la placa
- CS: chip select, activa o desactiva el display.
- C/D: command/data, determina si es un comando o un dato lo que se le envía.
- BUSY: ocupado, le dice a la placa si está disponible o no para la recepción de un byte.
- SCK: señal de reloj aportada por la placa.
- SI: serial input, pin por el que se le envía cada bit del dato/comando.
- SO: serial output, no lo usamos.

Como vemos, el envío de datos se realiza por un único pin serial, es decir, se van enviando los bits uno a uno hasta enviar el octavo bit. Es entonces cuando el chip del display codifica dicho byte y lo traduce a un carácter o un comando.

Además, para poder controlar en apagado y encendido del display lo hemos conectado a un circuito con un transistor como se muestra a continuación:



3. ANOTACIONES DEL CÓDIGO

3.1. NUESTRAS FUNCIONES DEFINIDAS

```
/*************************************************************************
 * mode: será 0 si queremos apagarlo y 1 si queremos encenderlo
 *************************************************************************/
void LCD_display_ON_OFF (int mode)
{
  if (mode == 0) {
    HAL_GPIO_WritePin(RESET_GPIO_Port, RESET_Pin, GPIO_PIN_SET);
  } else if (mode == 1) {
    HAL_GPIO_WritePin(RESET_GPIO_Port, RESET_Pin, GPIO_PIN_RESET);
  }
}
```

Este código únicamente nos sirve para apagar o encender el display dependiendo de si el modo es 0 (apagar) o 1 (encender). Y según el modo, activamos o desactivamos el pin.

Esta es la función más importante de nuestro proyecto. En ella sucede el envío de los 8 bits de dato ó comando al display.

Primero comienza poniendo a nivel bajo el pin de selección (funciona a nivel bajo) para el envío y posteriormente pregunta si está disponible. Cuando lo está comprueba si es un dato o comando lo que queremos enviar y comienza con el envío. Este se realiza bit a bit, y dependiendo de si el bit es 0 o 1, enviamos un 0 o 1 lógico. Todo esto siempre controlando la señal de reloj, como se muestra.

Por último volvemos a activar el pin de selección, para detener el envío.

```
/*************************************************************************
 * comando: será 1 si lo que se va a enviar al display es un comando
 * será 0 si lo que se va a enviar al display es un dato
 * byte: el comando o dato respectivo
 *************************************************************************/
void LCD_enviarByte(uint8_t comando, uint8_t byte)
{
  // Ponemos pin CHIPSEL a nivel bajo mientras enviamos comandos
    HAL_GPIO_WritePin(CHIPSEL_GPIO_Port, CHIPSEL_Pin, GPIO_PIN_RESET);

    // IMPORTANTE !! comprobar si está ocupado antes de enviar
    LCD_ocupado();

    // Comprobamos si se le envía comando o dato
    if (comando) {
        // Activamos pin para recepción de comando
        HAL_GPIO_WritePin(COMDAT_GPIO_Port, COMDAT_Pin, GPIO_PIN_SET);

    } else {
        // Activamos pin para recepción de dato
        HAL_GPIO_WritePin(COMDAT_GPIO_Port, COMDAT_Pin, GPIO_PIN_RESET);
    }

    // Enviamos comando o dato (bit a bit)

    for (int i = 1; i <= 8; i++) {
        // Aplicamos máscara para quedarnos con el msb


        if (byte & 0x80) {   // bit a 1
            HAL_GPIO_WritePin(DATSER_GPIO_Port, DATSER_Pin, GPIO_PIN_SET);
        } else {   // bit a 0
            HAL_GPIO_WritePin(DATSER_GPIO_Port, DATSER_Pin, GPIO_PIN_RESET);
        }

        HAL_GPIO_WritePin(SCK_GPIO_Port, SCK_Pin, GPIO_PIN_RESET);
        HAL_Delay(10);
        HAL_GPIO_WritePin(SCK_GPIO_Port, SCK_Pin, GPIO_PIN_SET);
        HAL_Delay(10);

        byte <<= 1;
    }

    // Ponemos pin CHIPSEL a nivel alto al finalizar el envío de comandos
    HAL_GPIO_WritePin(CHIPSEL_GPIO_Port, CHIPSEL_Pin, GPIO_PIN_SET);
}
```

```
/**************************************************************************
 * banco: sobre el cual se va a escribir por lo general el BANCO0
 * entrada_caracteres: si entran por la izquierda (0) o por la derecha (1)
 * al display. Por lo general comando SCMR.
 * La secuencia de incialización de comandos será:
 * Alimentar Display, SFF, BANCO, DISPLAYON, SCMR
 **************************************************************************/
void LCD_inicializacion(int banco, int entrada_caracteres)
{
  //Configuramos marco de frecuencia
  LCD_enviarByte(COMMAND, SFF);

  // Habilitamos banco: 0 -> banco 0;  1 -> banco 1;  2 -> bancos 0 y 1 (Todos en SYNC OUTPUT)
  // *** DUDA: usar banco0 = 0x1A y banco1= 0x1B ó ambos=0x1F *** //
  if (banco == 0) {
    LCD_enviarByte(COMMAND, BANCO0);
  }

  if (banco == 1) {
    LCD_enviarByte(COMMAND, BANCO1);
  }

  if (banco == 2) {
    LCD_enviarByte(COMMAND, BANCO_0_1);
  }

  // Encendemos display
  LCD_enviarByte(COMMAND, DISP_ON);

  LCD_enviarByte(COMMAND, SANDM);

  // Habilitamos escritura por la derecha
  if (entrada_caracteres == 0) {
    LCD_enviarByte(COMMAND, SCML);
  }

  if (entrada_caracteres == 1) {
    LCD_enviarByte(COMMAND, SCMR);
  }
}
```

En esta parte del código lo que se hace es inicializar el display. Para ello usamos la función anterior *LCD_enviarByte* para enviarle los bits. Como se puede observar, en este caso son todos comandos, que son los responsables de configurar el display.

Estos comandos se encuentran definidos en la cabecera *main.h* proporcionada junto al proyecto.

Esta función, que ya se nos daba implementada, lo único que hace es leer del pin BUSY, de manera que podamos saber cuándo está disponible el display para la recepción de datos o comandos.

```
/**************************************************************************
 * detectar si el display admite comandos o datos comprobando la
 * señal BUSY
 **************************************************************************/
void LCD_ocupado(void)
{
    while (!HAL_GPIO_ReadPin(BUSY_GPIO_Port, BUSY_Pin));   // GPI
}
```

3.2. NUESTRO BUCLE WHILE

```
  /* USER CODE BEGIN 2 */

  LCD_display_ON_OFF(0);
  HAL_Delay(3000);
  LCD_display_ON_OFF(1);

  LCD_inicializacion(2, 1);

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

      // Enviamos cadena como dato (DATA)
      HAL_Delay(10);
      LCD_enviarByte(DATA, 'A');
      HAL_Delay(10);
      LCD_enviarByte(DATA, 'L');
      HAL_Delay(10);
      LCD_enviarByte(DATA, 'O');
      HAL_Delay(10);
      LCD_enviarByte(DATA, 'H');

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

Por último, tenemos el programa principal. En él comencemos haciendo una prueba apagando y encendiendo el display, lo inicializamos y comenzamos con el bucle while. En este bucle lo que hacemos es ir enviando cada letra (se envía su carácter ascii) cada 10 milisegundos.